```java
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Map;

//IQ Frequency count (count the Frequency of each word)
public class HashMapDemo6
{
    public static void main(String[] args)
    {
       Map<String,Integer> frequency = new HashMap<>();

        String text = "java is fun, java is powerful java is great";
        String words[] = text.split(" ");

        for(String word : words)
        {
           frequency.put(word, frequency.getOrDefault(word, 0)+1);
        }

        IO.println(frequency);

    }
}
```

Map Methods :
----
1) default V computeIfAbsent(K key, Function<K,V> mappingFunction) //Insert & Compute

2) default V merge(K key, V value, BiFunction<K,V,V> mappingFunction) //Insert & merge the values

3) default V computeIfPresent(K key, BiFunction<K,V,V> mappingFunction) //Compute If key is already available
(Will not insert the entry)

default V computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)

Predefined method of Map interface added from JDK 1.8V as a default method

It is used to COMPUTE & INSERT the value in the Map by using given Function<K,V>, If key is not available in the Map collection.

Important Points related to computeIfAbsent() :

Key already has a value => Do nothing, return that existing value.

Key is new or null => Create a new value (using the Function) and put it in the map.

If the function gives null => Don't put anything in the Map.

```java
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Map;

//V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)

public class HashMapDemo7
{
   public static void main(String[] args)
   {
     HashMap<String,Integer> map = new HashMap<>();
     map.computeIfAbsent("Apple", key -> 10);
     map.computeIfAbsent("Apple", key -> 20);
     map.computeIfAbsent("Mango", key -> 50);
     map.computeIfAbsent("Grapes", key -> key.length());
     map.computeIfAbsent(null, key -> 100);
     map.computeIfAbsent("Kiwi", key -> 500);
     map.computeIfAbsent("Kiwi", key -> null);
     map.computeIfAbsent("Orange", key -> null);


     map.forEach((k,v)-> IO.println(k+" : "+v));



   }
}
```

---

default V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)

It adds the entry in the map, if the key is new, otherwise it combines old and new key value with the given function. If function result is null then key will be removed.It accepts entrire Map and perform operation based on the given key.

Different Cases :
----
a) If the key is missing OR has null [Insert the given value]
   map.merge("A", 10, (oldVal, newVal) -> oldVal + newVal);
   "A" not present in the map → map = {A=10}

b) If the key already has a value
  [Combine old value and new value using the function]
  map.merge("A", 5, (oldVal, newVal) -> oldVal + newVal);
  "A" was 10 → 10 + 5= 15 → map = {A=15}

c) If the function gives null [Remove the key from the map]
   map.merge("A", 5, (oldVal, newVal) -> null);
   Removes "A" from the Map. [because key "A" is already available in the Map]

d) If key is not present and Function returns null then key will be inserted
   map.merge("Kiwi", 25, (oldValue, newValue)-> null);
   Note : Kiwi is not present as a key then It will be inserted.

```java
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Map;

/*default V merge
    (K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)  */

public class HashMapDemo8
{
public static void main(String[] args)
{
      Map<String, Integer> map = new HashMap<>();
      map.merge("Pencil",10, (oldValue, newValue) -> oldValue);
      map.merge("Pencil",20, (oldValue, newValue) -> newValue);
      map.merge("Pencil",40, (oldValue, newValue) -> oldValue + newValue);

      map.merge("Eraser", 5, (oldValue, newValue) -> oldValue);
      map.merge("Eraser", 3, (oldValue, newValue) -> oldValue - newValue);

      map.merge(null, 10, (oldValue, newValue)-> newValue);



      map.merge("Pen", 100, (oldValue, newValue)-> oldValue);
      //Key (Pen) is  present and function is returning null so It will be deleted
      map.merge("Pen", 100, (oldValue, newValue)-> null);


      //Key(Sharpener) is absent and function is returning null so will be inserted
      map.merge("Sharpener", 100, (oldValue, newValue) -> null);


      map.forEach((k,v)-> IO.println(k+" : "+v));
   }
}
```

---

default V computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)

It changes the value only if the key is already present in the map. It will not insert the entry in the Map. If Function returns null, the key is removed from the Map.

Different Cases :
----
a) If the key exists in the map and its value is not null [Update the value using the function]

b) If the key is missing OR value is null [Do nothing]

c) If the function returns null [Remove the key]

```java
package com.ravi.hash_map;

import java.util.HashMap;

//default V computeIfPresent
//(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)

public class HashMapDemo9
{
    public static void main(String[] args)
    {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("Java", 10);
        map.put("Python", 9);
        map.put("React", 5);
        map.put("HTML", 7);

        map.computeIfPresent("Java", (k,v)-> k.length()+ v);
        map.computeIfPresent("Python", (k,v)-> 5);
        map.computeIfPresent("React", (k,v)-> 5+5);
        map.computeIfPresent("HTML", (k,v)-> null);


        IO.println(map);
    }

}
```

Map with List of values [Map<K, List<V>>]

```java
package com.ravi.hash_map;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

//Program where a Student can select multiple subjects according to the Choice

public class HashMapDemo10
{
    public static void main(String[] args)
    {
       Map<String,List<String>> map = new HashMap<>();
       map.computeIfAbsent("Scott", key -> new ArrayList<String>()).add("Java");
       map.computeIfAbsent("Scott", key -> new ArrayList<String>()).add("Adv Java");

       map.computeIfAbsent("Alen", key -> new ArrayList<String>()).add("HTML");
       map.computeIfAbsent("Alen", key -> new ArrayList<String>()).add("JavaScript");


       map.computeIfPresent("Scott", (key, value)->
       {
          value.add("Spring");
          return value;
       });

       map.computeIfPresent("Alen", (k,v)->
       {
         v.add("React");
         return v;
       });

       map.forEach((k,v)-> IO.println(k+" : "+v));

    }
}
```

---

public static <T> T min(Collection<? extends T> coll, Comparator<? super T> comp) :

It is a static utility method in the java.util.Collections class.

It is used to find the minimum element in a Collection based on a custom Comparator provided as a 2nd parameter.

The first parameter accepts Collection as a parameter

Note : We have also max() method.

```java
package com.ravi.hash_map;

import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.Map.Entry;

//cheapest flight

public class HashMapDemo11
{
  public static void main(String[] args)
  {
      Map<String, Integer> flights = new HashMap<>();
      flights.put("Mumbai", 5500);
      flights.put("Bangalore", 7000);
      flights.put("Chennai", 6500);
      flights.put("Vizag", 5000);
      flights.put("GOA", 4500);

      Entry<String, Integer> min = Collections.min(flights.entrySet(),(f1,f2)->
f1.getValue().compareTo(f2.getValue()));

      IO.println("cheapest flight from Hyderabad to "+min.getKey()+" is available at RS
:"+min.getValue());

      Map<String, Integer> allflights = new HashMap<>();
      allflights.put("Mumbai", 5500);
      allflights.put("Bangalore", 7000);
      allflights.put("Chennai", 6500);
      allflights.put("Vizag", 7000);
      allflights.put("GOA", 4500);
      allflights.put("Delhi", 7000);

      Collection<Integer> allPrices = allflights.values();

      Integer maxPrice = Collections.max(allPrices);

      for(Map.Entry<String, Integer> map : allflights.entrySet())
      {
          if(map.getValue().equals(maxPrice))
          {
              IO.println("Expensive flights from Hyderabad to "+map.getKey()+" is available at RS
:"+map.getValue());
          }
      }



  }
}
```